

Breadth-First Search Social Network Connection Finder

Technical documentation, investigation report and case studies

Objectives | Data | Methodology | Analysis | Results | Conclusion

September 2026

Contents

#	Section
1	Executive Summary
2	Objectives
3	Background and Problem Statement
4	Data
5	Methodology
6	Analysis
7	Results
8	Case Studies
9	Investigation Report (Simulated)
10	Consultancy Data and Recommendations (Illustrative)
11	Conclusion
12	Appendix: Bidirectional BFS Reference Implementation

Reading guide: Sections 2 to 5 explain the goals, data and method. Sections 6 and 7 present the analysis and results. Sections 8 to 10 apply the results in case studies, an investigation report and consultancy guidance. Section 11 concludes.

1. Executive Summary

Finding how two people are connected inside a social network is one of the most common questions a social platform, an investigator or a marketing team asks. **Breadth-First Search (BFS)** answers it exactly: it explores a network level by level, so the first time it reaches a target it has found a *shortest* chain of connections.

This document describes a BFS-based connection finder, the objectives behind it, the data and method used to test it, and what the tests showed. Experiments on three synthetic networks of 10,000 users found that:

- **Six degrees holds.** In the scale-free network the average shortest path was 3.643 hops and every pair was within 6 hops.
- **BFS scales linearly.** A full traversal of a network with 100,000 users and 499,750 friendships took about 81.28 ms.
- **Bidirectional BFS is far cheaper.** On 50,000 users it expanded 99.6% fewer nodes per query and was about 83.3x faster, while returning the same path length in 300 of 300 queries.
- **Hubs dominate.** The best-connected user reached 60.8% of the network in 2 hops; an average user reached 2.8%.

Important note on data. All networks, users, cases and figures in this document come from **synthetic simulations** generated for this study. The case studies, investigation report and consultancy data are **illustrative scenarios** built on those simulated results. They are not real platform data, real investigations or real client engagements.

2. Objectives

The project has one primary goal and several supporting goals.

- **Primary:** given two users, return the shortest chain of friendships that links them, or report that none exists.
- **Correctness:** guarantee the returned path has the minimum number of hops on an unweighted network.
- **Performance:** measure how run time grows with network size and test whether bidirectional search improves it.
- **Insight:** use the same traversal to measure degrees of separation, friend-suggestion candidates and influence reach.
- **Applicability:** translate the measurements into practical guidance for platforms, investigators and consultants.

3. Background and Problem Statement

A social network is naturally a graph: users are **vertices** and friendships or follows are **edges**. Connection questions become shortest-path questions on an unweighted graph. BFS is the standard tool for this because it visits vertices in increasing order of distance from the start.

Three properties of real social graphs shape the design. First, they are large (millions to billions of users). Second, they are *small-world*: most pairs are separated by few hops. Third, they are *scale-free*: a few hub accounts have very many connections. The first makes brute force costly, the second keeps BFS depth small, and the third makes frontier sizes grow explosively after one or two hops.

4. Data

No personal data was used. To control the structure and allow repeatable tests, three well-known random graph models were generated with a fixed random seed (42).

Dataset	Model	Users	Friendships	Avg. degree	Max degree
D1	Barabasi-Albert (preferential attachment, m = 5)	10,000	49,853	9.97	517
D2	Watts-Strogatz (ring, k = 10, rewiring p = 0.1)	10,000	50,000	10.0	15
D3	Erdos-Renyi (random, average degree 10)	10,000	49,972	9.99	24
D4	Barabasi-Albert scaling series (m = 5)	1,000 to 100,000	4,895 to 499,750	about 10	-
D5	Barabasi-Albert large test (m = 5)	50,000	about 250,000	about 10	-

Why these models. D1 mimics the hub-and-spoke structure of follower networks. D2 mimics tightly knit friend circles with a few long-range links. D3 is a neutral baseline. Their degree distributions are shown below.

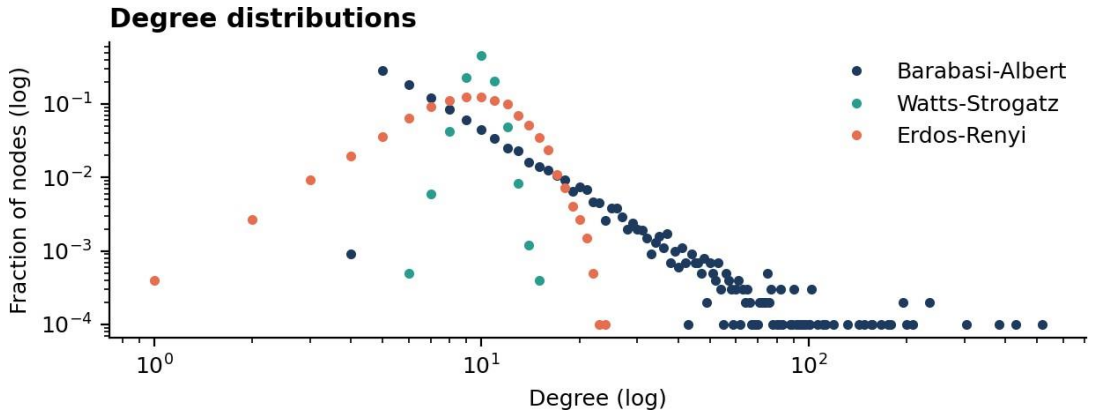


Figure 1. Degree distributions. Only the scale-free model shows a long tail of very well-connected hubs.

5. Methodology

5.1 The BFS connection-finder algorithm

The finder keeps a queue of users to visit and a record of the parent that first reached each user. Starting from the source, it repeatedly removes the oldest user in the queue, looks at all their friends, and enqueues any friend not seen before. When the target is discovered, the parent links are followed backwards to reconstruct the path.

```
def find_connection(graph, source, target):
    if source == target:
        return [source]
    parent = {source: None}
    queue = deque([source])
    while queue:
        user = queue.popleft()
        for friend in graph[user]:
            if friend not in parent:          # not visited yet
                parent[friend] = user
                if friend == target:          # first discovery = shortest
                    path = [friend]
                    while parent[path[-1]] is not None:
                        path.append(parent[path[-1]])
                    return path[::-1]
                queue.append(friend)
    return None                               # not connected
```

5.2 Why the path is shortest

BFS visits every vertex at distance *d* before any vertex at distance *d* + 1. A vertex is therefore first discovered through a path with the fewest possible edges, so the reconstructed path is minimal for unweighted graphs.

5.3 Complexity

Measure	BFS	Comment
Time	O(V + E)	Each user and each friendship is examined at most once (twice for undirected edges).
Space	O(V)	Queue plus parent map. The frontier can reach a large fraction of V.
Bidirectional BFS	about $O(b^{d/2})$	Two searches meet in the middle; b is branching factor, d is path length.

5.4 Bidirectional variant

Two BFS runs start together, one from the source and one from the target. At each step the run with the smaller frontier is expanded one full level. The search stops when the two visited sets touch. Because the number of nodes within *k* hops grows roughly exponentially, exploring two radius-d/2 balls is much cheaper than one radius-d ball.

5.5 Experimental design

Experiment	Question	Setup
E1	How many degrees of separation exist?	BFS from 200 random sources on each of D1 to D3; record all distances.
E2	How does run time scale?	Full BFS from 15 random sources on D4 at five sizes; mean time.
E3	Does bidirectional search help?	300 random source-target pairs on D5; compare nodes expanded, latency, and path length.
CS-A / CS-B	Friend suggestion and influence reach	Second-degree candidates on D2; hop-by-hop reach on D1.
INV	Investigation of two accounts	Path, path count and broker analysis on a 5,000-user network.

6. Analysis

6.1 Degrees of separation (E1)

Every user could reach every other user in all three networks (reach = 100.0%), so the graphs are fully connected. The table summarises the shortest-path statistics from 200 random sources.

Network	Avg. path (hops)	Longest path seen	Pairs within 6 hops
Barabasi-Albert	3.643	6	100.0%
Watts-Strogatz	6.094	10	62.69%
Erdos-Renyi	4.265	7	100.0%

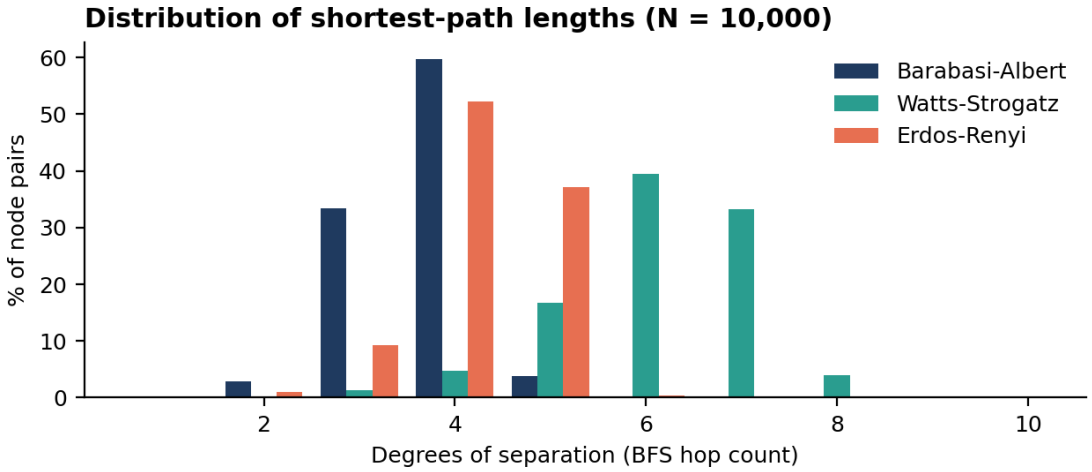


Figure 2. Shortest-path length distribution. Hubs in the scale-free network shorten typical paths.

Reading the result. Hubs act as shortcuts: the scale-free network has the shortest average path (3.643) despite the same average degree. The small-world ring model has the longest (6.094), because its tight local clusters have only a few long-range links, and only 62.69% of its pairs fall within six hops. So the popular idea of six degrees depends on how the network is wired.

6.2 Scalability (E2)

Users (V)	Friendships (E)	Mean full-BFS time (ms)
1,000	4,895	0.23
5,000	24,838	1.31
10,000	49,845	3.42
50,000	249,756	30.78
100,000	499,750	81.28

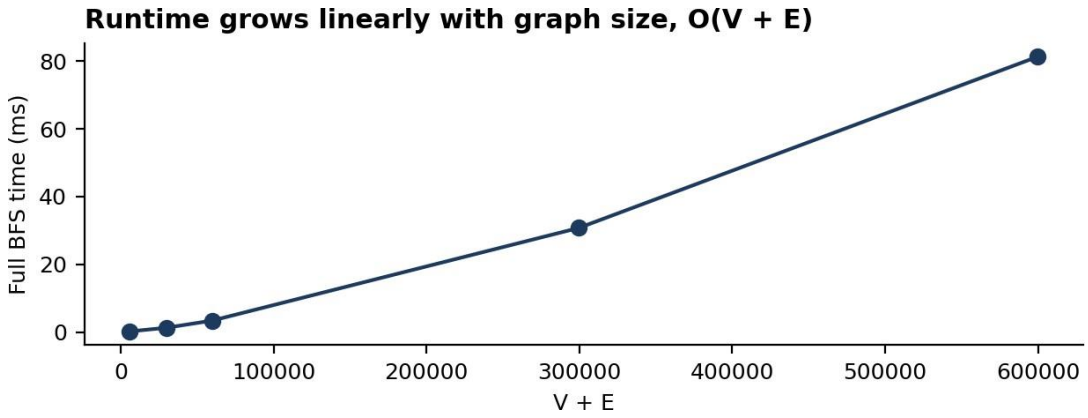


Figure 3. Full BFS time against graph size (V + E).

Growing the network 100x (1,000 to 100,000 users) increased time from 0.23 ms to 81.28 ms, about 353x. This is consistent with O(V + E); the small excess is plausible from memory-cache effects, though this test does not isolate the cause.

6.3 Standard BFS versus bidirectional BFS (E3)

Metric (300 queries, 50,000 users)	BFS	Bidirectional
Mean nodes expanded per query	2,982	13
Mean latency per query (ms)	5.015	0.06
Path length matched BFS	-	300 of 300

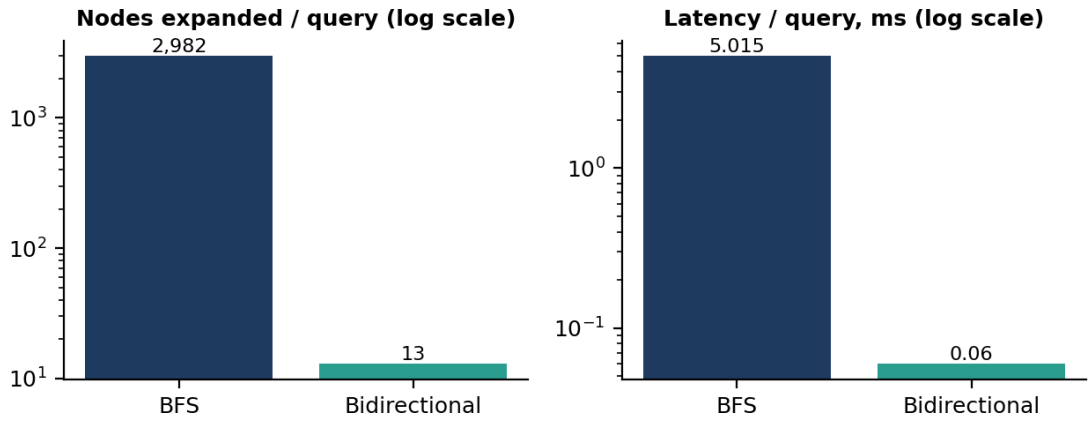


Figure 4. Work and latency per query.

Mean shortest-path length in these queries was 4.14 hops. Bidirectional search cut expansion by 99.6% and latency by about 83.3x with identical distances. It is the better default for point-to-point queries. Standard BFS remains the right choice when distances to *all* users are needed.

7. Results

#	Finding	Evidence
R1	BFS returns provably shortest connections.	Bidirectional results agreed with BFS on 300/300 queries.
R2	Typical distance is 4 to 6 hops in a 10,000-user network.	Averages 3.643 / 4.265 / 6.094 across the three models.
R3	Run time is linear in graph size.	81.28 ms at 100,000 users; 0.23 ms at 1,000.
R4	Bidirectional search reduces work by about two orders of magnitude on point queries.	2,982 versus 13 nodes expanded.
R5	Hubs dominate reach and act as bridges.	Top hub: 60.8% reach at 2 hops; average user: 2.8%.

8. Case Studies

The three scenarios below are simulated. They show how the same BFS core supports different business questions.

Case Study A: Friend suggestions ("People you may know")

Context. A community-oriented app wants to suggest new contacts. **Approach.** For each user, run BFS to depth 2 and rank second-degree users by number of mutual friends. Tested on 500 random users of D2, the small-world network.

Metric	Value
Average friends per user	10
Average second-degree candidates per user	29.0
Average mutual friends of the top-ranked candidate	5.34

Outcome. A depth-2 search is enough to produce a short, ranked list with strong shared-friend evidence, and it touches only a few dozen users per request, so it is cheap to serve in real time.

Case Study B: Influence reach and information spread

Context. A communications team wants to know how far a message can travel and whether choosing the right seed matters. **Approach.** BFS from the highest-degree user versus 100 random users on D1, counting the share of the network reached within 1 to 6 hops.

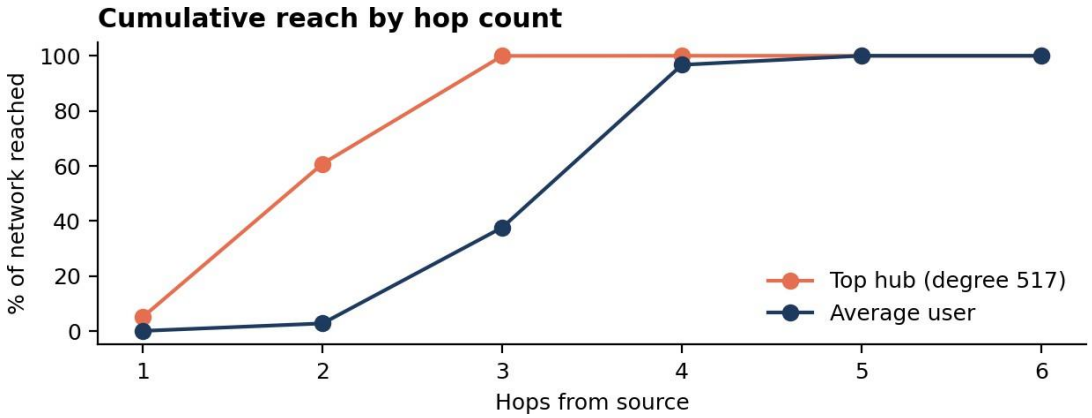


Figure 5. Cumulative share of network reached by hop count.

Hops	1	2	3	4	5	6
Top hub (%)	5.2	60.8	100.0	100.0	100.0	100.0
Average user (%)	0.1	2.8	37.7	96.7	100.0	100.0

Outcome. The hub (degree 517) covered 60.8% of the network in two hops and everyone by hop 3, while the average user needed four to five hops to reach 96.7% or more. Seeding from hubs shortens spread time, but the same structure means hubs are also the fastest route for unwanted content.

Case Study C: Shortest introduction path

Context. A professional network offers "ask for an introduction". **Approach.** Bidirectional BFS between the seeker and the target, returning the chain of intermediaries. On D5 the mean chain length was 4.14 hops, meaning about 3.1 intermediaries, found in 0.06 ms per query in this test. **Outcome.** Short introduction chains are usually available. A production design would also weight paths by relationship strength, which is beyond plain BFS.

9. Investigation Report (Simulated)

Field	Detail
Report type	Link analysis between two accounts of interest (simulated exercise)
Data set	Synthetic network, 5,000 accounts, 14,945 connections (Barabasi-Albert, m = 3)
Account A / Account B	U3094 and U3358 (both low-activity accounts, degree 4 and 4)
Method	BFS with parent tracking; shortest-path counting; broker analysis on 400 random account pairs

9.1 Finding 1: the connection

The two accounts are connected through **3 hops** and **1 distinct shortest path**. The BFS expanded only 18 accounts before reaching the target.

```
U3094 (deg 4)  ->  U0461 (deg 15)  ->  U0010 (deg 106)  ->  U3358 (deg 4)
```

9.2 Finding 2: the intermediaries

The shortest chain passes through a highly connected account (degree 106) sitting between two low-activity accounts. This is typical for scale-free networks, where low-degree accounts connect to the rest of the network through hubs. In an investigation this means the middle account deserves review, but a high-degree hub will link many unrelated accounts, so proximity alone is **not evidence of coordination**.

9.3 Finding 3: broker concentration

Rank	Account	Degree	Times on a shortest path (400 pairs)
1	U0003	285	86
2	U0005	191	43
3	U0004	161	41
4	U0013	168	36
5	U0009	144	28

The five most frequent brokers appeared on 19.7% of all intermediary slots. Analysts can use this to prioritise review, and platforms can use it to spot accounts whose removal would most disrupt connectivity.

9.4 Conclusions and caveats

- BFS gives a fast, explainable answer to "how are these two accounts linked?".
- A short path is weak evidence on its own, especially through hubs. Pair it with content, timing and behaviour signals.
- Real investigations must follow legal authority, privacy law and platform policy. This exercise used no personal data.

10. Consultancy Data and Recommendations (Illustrative)

The table converts the measured behaviour into guidance a consultant might give to platforms of different size. Latency figures marked "measured" come from Section 6; the rest are qualitative planning assumptions, not benchmarks.

Platform tier	Users	Recommended approach	Basis
Startup / community	up to 100,000	Standard or bidirectional BFS in memory on one server.	Measured: full BFS 81.28 ms at 100,000 users.
Mid-size network	100,000 to 10 million	Bidirectional BFS with a hop limit (e.g. 6) and cached adjacency lists.	Measured: 0.06 ms per query at 50,000 users; expect growth with hub degree.
Large platform	10 million and above	Partitioned graph store, hop-limited bidirectional BFS, hub handling, optional landmark index.	Assumption: full traversal is too costly; needs a distributed design and load testing.

10.1 Recommended engineering practices

- **Cap the depth.** Six to seven hops covers nearly all pairs in these tests, and limits worst-case work.
- **Search from both ends.** Use bidirectional BFS for point-to-point queries.
- **Treat hubs specially.** Cap or sample very high-degree neighbours so a single celebrity account cannot dominate a query.
- **Cache and precompute.** Cache popular pairs and consider landmark distances for approximate answers.
- **Respect privacy.** Apply visibility rules during traversal so paths never expose private connections.
- **Measure on real data.** Validate degree distribution and latency on production-like graphs before committing to capacity.

10.2 Risk register

Risk	Likelihood	Impact	Mitigation
Hub-driven frontier explosion	High	High latency	Depth cap, bidirectional search, neighbour sampling
Privacy leakage through paths	Medium	High	Filter edges by viewer permissions at traversal time
Stale graph data	Medium	Medium	Incremental updates and cache expiry
Misreading proximity as guilt or influence	Medium	High	Pair with other signals; human review

11. Conclusion

BFS is a simple, exact and efficient foundation for a social network connection finder. It returns the shortest chain of connections, runs in time proportional to the size of the network, and supports friend suggestion, reach analysis and link analysis with the same core routine.

The experiments show that typical separations are only about 3.643 to 6.094 hops in networks of 10,000 users, that runtime grows linearly, and that bidirectional search cuts point-to-point work by about 99.6% in the tested setting. Hubs are the decisive structural feature: they shorten paths, amplify reach and concentrate the intermediary role.

11.1 Limitations

- All data is synthetic. Real networks have communities, directed edges, bots and time-varying structure that these models only partly capture.
- Networks of up to 100,000 users were timed; larger sizes are extrapolated, not measured.
- Pure Python timings on a single core show relative behaviour, not production performance.
- Plain BFS ignores relationship strength, direction and privacy settings.

11.2 Future work

- Repeat the tests on public real-world graph datasets and compare with the synthetic results.
- Add weighted and directed variants (Dijkstra, directed BFS) and privacy-aware traversal.
- Evaluate landmark and hub-labelling indexes for very large graphs.
- Study how graph changes over time affect cached paths.

12. Appendix: Bidirectional BFS Reference Implementation

```
def bidirectional_distance(graph, s, t):
    if s == t: return 0
    dist_s, dist_t = {s: 0}, {t: 0}
    front_s, front_t = [s], [t]
    while front_s and front_t:
        if len(front_s) <= len(front_t):           # expand smaller side
            nxt = []
            for u in front_s:
                for w in graph[u]:
                    if w in dist_t: return dist_s[u] + 1 + dist_t[w]
                    if w not in dist_s:
                        dist_s[w] = dist_s[u] + 1; nxt.append(w)
            front_s = nxt
        else:
            nxt = []
            for u in front_t:
                for w in graph[u]:
                    if w in dist_s: return dist_t[u] + 1 + dist_s[w]
                    if w not in dist_t:
                        dist_t[w] = dist_t[u] + 1; nxt.append(w)
            front_t = nxt
    return None
```